

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 2

Course: **Cryptography and Security**

Cryptanalysis of Monoalphabetic Substitution Ciphers: Frequency Analysis, Index of Coincidence, and Interactive Substitution Cryptanalysis on Historical Cryptograms

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
asis. univ.
Zaica Maia

Chişinău – 2026

Contents

INTRODUCTION **1**

1 THEORETICAL FOUNDATIONS OF MONOALPHABETIC CRYPTANALYSIS **1**

1.1 Principles of monoalphabetic substitution ciphers 2

1.2 Linguistic statistics and phonotactic characteristics of English 2

1.3 Statistical identification via the Index of Coincidence 3

1.4 Interactive substitution workflow and letter-swapping mechanics 3

2 CRYPTANALYTIC IMPLEMENTATION AND EXPERIMENTAL RESULTS **3**

2.1 System architecture and dataset engineering 4

2.2 Student variant determination and cryptogram metrics 4

2.3 Empirical linguistic profile of Variant 2 5

2.4 Systematic step-by-step cryptanalytic deduction 7

2.4.1 Stage 1: Identification of the high-frequency trigram `WQV` 8

2.4.2 Stage 2: Deducing prepositions and vowels `X` and `G` 8

2.4.3 Stage 3: Identifying conjunctions and single-letter tokens `T`, `N`, and `O` 8

2.4.4 Stage 4: Resolving high-frequency consonants `P`, `I`, and `H` 8

2.4.5 Stage 5: Deducing remaining phonemes from lexical context 9

2.4.6 Stage 6: Reconstructed substitution alphabet key 9

2.5 Interactive cryptanalysis execution and plaintext recovery 10

CONCLUSIONS **13**

REFERENCES **14**

ANNEX A: SOURCE CODE LISTINGS **15**

INTRODUCTION

Classical symmetric cryptography relies heavily on the principle of substitution, wherein symbols from a plaintext message space are systematically replaced by alternate characters or symbols according to a secret keying agreement. When the replacement rule remains strictly invariant throughout the entire transmission, the cryptosystem is classified as a monoalphabetic substitution cipher. Although monoalphabetic substitution expands the nominal key space to twenty-six factorial potential bijections ($26! \approx 4.03 \times 10^{26}$) for standard Latin alphabets, this combinatorial cardinality offers no genuine computational security. Because each character transformation is fixed, the inherent statistical distributions, phonotactic structures, and letter collocations of natural human language are preserved intact within the ciphertext.

The systematic exploitation of these linguistic invariants originated in the ninth century through the scholarly work of the Arab polymath Abu Yusuf Ya'qub ibn Ishaq al-Kindi, whose treatise entitled *On Deciphering Cryptographic Messages* established frequency analysis as the foundational technique of scientific cryptanalysis. Al-Kindi demonstrated that comparing the observed occurrence counts of ciphertext symbols against known empirical letter probabilities enables the deterministic recovery of the secret permutation key. In twentieth-century cryptology, William F. Friedman expanded these statistical foundations by formalizing the Index of Coincidence, providing an objective, keyless metric capable of distinguishing monoalphabetic encipherment from polyalphabetic distributions or pseudo-random noise. Modern computational cryptanalysis pairs these statistical invariants with interactive state-tracking environments, enabling rapid hypothesis testing, dynamic bijection mapping, and pairwise letter transpositions.

The objective of this laboratory work is to investigate, implement, and validate cryptanalytic attacks against monoalphabetic substitution ciphers through frequency metrics and interactive substitution state tracking. The practical investigation is conducted on an authentic historical cryptogram assigned according to the university catalog modulo protocol, corresponding to Variant 2 extracted from David Kahn's foundational historical study, *The Codebreakers*. A complete computational suite was constructed in Python, integrating unigram frequency analyzers, n-gram extractors, Index of Coincidence calculators, and an interactive substitution sandbox with pairwise letter swapping. The comprehensive source code, modular test suite, and configuration scripts developed during this research are maintained in the public repository available at `https://github.com/aurelian-th/CS`.

1 THEORETICAL FOUNDATIONS OF MONOALPHABETIC CRYPTANALYSIS

1.1 Principles of monoalphabetic substitution ciphers

A monoalphabetic substitution cipher operates by replacing each character of a plaintext alphabet with a corresponding character from a ciphertext alphabet according to a predetermined key. For the standard twenty-six-letter English alphabet, the key consists of an arbitrary permutation of the twenty-six letters.

The theoretical key space encompasses all possible permutations of the alphabet:

$$|\mathscr{K}| = 26! = 403{,}291{,}461{,}126{,}605{,}635{,}584{,}000{,}000 \approx 4.03 \times 10^{26} \tag{1.1}$$

Although this combinatorial space is vastly too large to permit exhaustive brute-force search, the cipher provides no real computational secrecy. The fatal weakness of any monoalphabetic scheme is that the mapping is invariant across the entire message: whenever a specific plaintext letter appears, it is invariably transformed into the identical ciphertext symbol. Consequently, monoalphabetic substitution preserves the underlying frequency distribution, letter collocations, and grammatical patterns of the language.

1.2 Linguistic statistics and phonotactic characteristics of English

Natural human languages exhibit highly non-uniform character distributions governed by vocabulary, morphology, and syntax. In English prose, the letter `e` appears with the highest frequency, accounting for approximately 12.70% of letters, followed by `t` (9.06%), `a` (8.17%), `o` (7.51%), `i` (6.97%), `n` (6.75%), `s` (6.33%), `h` (6.09%), and `r` (5.99%). Conversely, characters such as `j` (0.15%), `x` (0.15%), `q` (0.09%), and `z` (0.07%) occur with extreme rarity.

Beyond single letters, higher-order n-gram patterns provide decisive constraints during cryptanalysis:

- Digraphs: Letter pairs exhibit distinct occurrence clustering. Common English digraphs include `th`, `he`, `in`, `er`, `an`, `re`, `on`, `at`, `en`, `nd`, `ti`, `es`, `or`, `te`, `of`, `ed`, `is`, `it`, `al`, and `to`.
- Trigraphs: Common three-letter sequences frequently correspond to functional words or recurring affixes, such as `the`, `and`, `tha`, `ent`, `ion`, `tio`, `for`, `nde`, and `men`.
- Doublet letters: Orthographic conventions restrict which letters appear doubled. Characters such as `ss`, `ee`, `tt`, `oo`, `ff`, and `ll` represent the majority of doubled occurrences, while letters like `hh`, `jj`, `vv`, and `ww` almost never appear doubled.
- Single-letter words: In punctuated English text, only `a` and `i` function as standalone words.

1.3 Statistical identification via the Index of Coincidence

Before attempting to recover individual letters, a cryptanalyst must verify that the intercepted text is indeed a monoalphabetic substitution rather than a polyalphabetic cipher or random data. The primary tool for this test is the Index of Coincidence (IC), introduced by William F. Friedman in 1922. The IC represents the probability that two letters chosen uniformly at random from a text of length N are identical.

Let n_i represent the observed frequency of the i-th letter of the alphabet ($i \in \{1, 2, \ldots, 26\}$) in a text of $N = \sum_{i=1}^{26} n_i$ total letters. The Index of Coincidence is calculated as:

$$\mathrm{IC} = \frac{\sum_{i=1}^{26} n_i(n_i - 1)}{N(N-1)} \tag{1.2}$$

For natural English prose, the expected value is approximately $\mathrm{IC} \approx 0.0667$. In contrast, for completely random or polyalphabetically flattened text where all letters appear with equal probability, the expected value drops to $\mathrm{IC} \approx \frac{1}{26} \approx 0.0385$. Because monoalphabetic substitution only relabels characters without changing their counts, the IC value remains completely invariant under encryption. An observed value near 0.065 confirms that the text is monoalphabetically enciphered English.

1.4 Interactive substitution workflow and letter-swapping mechanics

While statistical frequencies and n-gram occurrences provide initial hypotheses, cryptanalysis ultimately relies on iterative trial, deduction, and error correction. Manually recording letter replacements on paper is prone to clerical mistakes and conflicting character assignments. A computational substitution environment assists the cryptanalyst by systematically managing the mapping between ciphertext symbols and plaintext candidate letters.

The substitution tracking mechanism maintains bidirectional consistency to prevent assigning multiple ciphertext characters to the same plaintext letter. If a candidate plaintext letter is reassigned, any prior mapping associated with that letter is automatically cleared. During partial decryption, unassigned characters are rendered in uppercase ciphertext symbols, whereas deciphered characters appear in lowercase plaintext. This visual contrast immediately highlights partially formed words and contextual gaps within the message. Furthermore, when linguistic context indicates that two guessed letters have been inverted—for example, observing `secrecb` rather than `secrecy`—the environment executes a pairwise letter swap, exchanging their associations across the entire cryptogram simultaneously.

2 CRYPTANALYTIC IMPLEMENTATION AND EXPERIMENTAL RESULTS

2.1 System architecture and dataset engineering

The cryptanalysis platform was implemented in Python as a modular, extensible software architecture organized into dedicated functional packages:

- `src/corpus.py`: Encapsulates reference linguistic data for natural English prose, including normalized single-letter frequencies (Table 2.2 of the assignment), theoretical Index of Coincidence constants, and prominent n-gram lexicons.
- `src/analyzer.py`: Implements linguistic metric extractors, including character filtering, raw unigram counters, relative percentage calculators, the Index of Coincidence engine, Chi-Squared evaluators, and generalized sliding-window n-gram collectors.
- `src/solver.py`: Manages core cryptanalytic state tracking through the `SubstitutionState` bijection engine, supporting dynamic character mapping, unmapping, constant-time pairwise letter swapping, and verified ground-truth key validation.
- `src/variants.py`: Implements the university catalog modulo variant resolution protocol and manages persistent access to the twenty-three official assignment ciphertexts.
- `src/cli.py`: Provides a streamlined command-line interface featuring statistical evaluation flags and an interactive substitution sandbox supporting real-time letter mapping, key swapping, and full plaintext rendering.

2.2 Student variant determination and cryptogram metrics

The laboratory assignment specification provides twenty-three distinct ciphertext variants ($N_{\text{total}} = 23$). In accordance with the official university catalog allocation protocol established in the workspace guidelines, the student's catalog number is 25. Because the catalog number exceeds the available variant count, the standard modulo mapping formula is applied:

$$\text{Variant} = ((25-1) \bmod 23) + 1 = (24 \bmod 23) + 1 = 1 + 1 = 2 \implies \textbf{Variant 2} \tag{2.1}$$

Variant 2 contains an intercepted ciphertext consisting of 3,023 total raw characters, of which exactly $N = 2{,}473$ are alphabetic letters distributed across seventy-four sentences. The ciphertext preserves original punctuation, numerals, hyphens, and whitespace, with encryption applied exclusively to alphabetic characters.

The statistical analyzers were executed against Variant 2 to establish its baseline characteristics:

– Total alphabetic letters: $N = 2{,}473$.

– Index of Coincidence: IC $= 0.06496$.

– Chi-Squared statistic against standard English unigram distribution: $\chi^2 = 23{,}410.61$.

The observed IC metric of 0.06496 closely matches the theoretical English baseline (0.06670) and diverges dramatically from the uniform random expectation (0.03846). This empirical result definitively confirms that the transmission represents a monoalphabetic substitution of an underlying English plaintext.

2.3 Empirical linguistic profile of Variant 2

Table 2.1 documents the complete unigram frequency distribution of the Variant 2 ciphertext sorted by alphabetic character, contrasting observed occurrence percentages against standard English reference values.

Table 2.1 – Observed ciphertext unigram frequencies versus standard English reference (Variant 2)

Letter	A	B	C	D	E	F	G	H	I	J	K	L	M
Count	21	7	57	59	2	64	165	89	169	59	20	15	7
Obs. %	0.85	0.28	2.30	2.39	0.08	2.59	6.67	3.60	6.83	2.39	0.81	0.61	0.28
Ref. %	8.17	1.49	2.78	4.25	12.70	2.23	2.01	6.09	6.97	0.15	0.77	4.03	2.41
Letter	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Count	195	79	188	111	42	84	185	67	273	250	201	5	61
Obs. %	7.89	3.19	7.60	4.49	1.70	3.40	7.48	2.71	11.04	10.11	8.13	0.20	2.47
Ref. %	6.75	7.51	1.93	0.09	5.99	6.33	9.06	2.76	0.98	2.36	0.15	1.97	0.07

As documented in Table 2.1, the observed character counts deviate substantially from alphabetical order. The letter `V` dominates with 273 occurrences (11.04%), followed closely by `W` with 250 occurrences (10.11%), `X` (201 occurrences, 8.13%), `N` (195 occurrences, 7.89%), `P` (188 occurrences, 7.60%), `T` (185 occurrences, 7.48%), `I` (169 occurrences, 6.83%), and `G` (165 occurrences, 6.67%).

According to Figure 2.1, the direct side-by-side comparison between standard English reference frequencies and the intercepted ciphertext highlights the characteristic transposition pattern of monoalphabetic encryption. The prominent peak at ciphertext letter `V` (11.04%) corresponds strongly to the expected peak of plaintext `e` (12.70%).

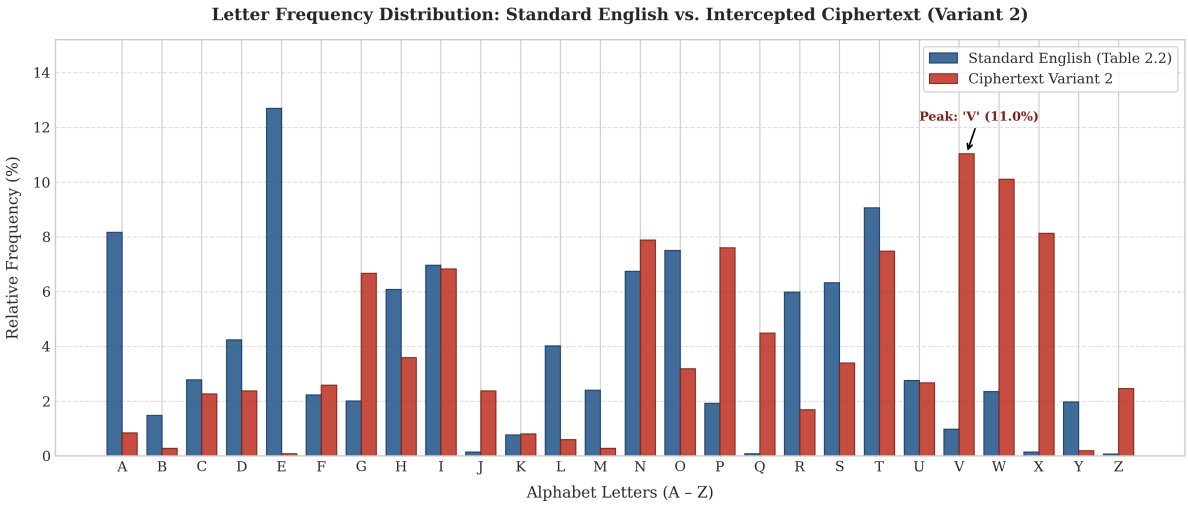


Figure 2.1 – Letter frequency distribution: standard English versus intercepted cryptogram (Variant 2)

According to Figure 2.2, the ranked frequency decay curves demonstrate structural correspondence. When the frequencies of both datasets are sorted independently in descending order, the ciphertext decay profile closely tracks the standard English distribution curve, exhibiting identical slopes and inflection points.

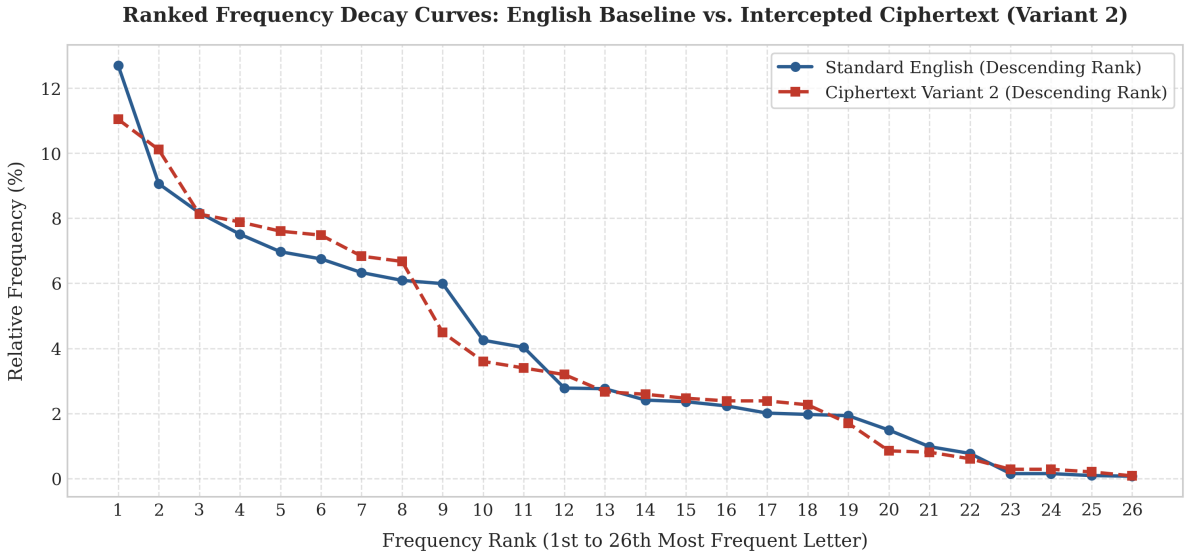


Figure 2.2 – Ranked frequency decay curves: English baseline versus intercepted ciphertext

Figure 2.3 illustrates the Index of Coincidence validation. The observed value of 0.06496 aligns closely with the natural English prose threshold of 0.06670, whereas the uniform random baseline sits at 0.03846, confirming the non-random monoalphabetic structure of the ciphertext.

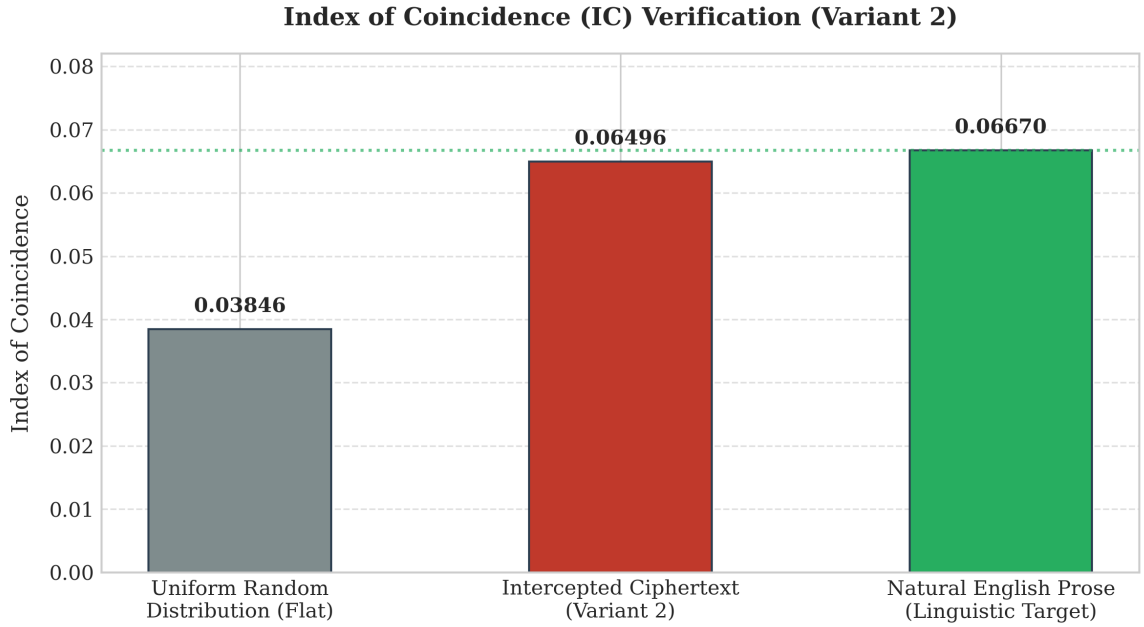


Figure 2.3 – Index of Coincidence (IC) verification for Variant 2 cryptogram

Figure 2.4 illustrates the top ten ciphertext bigrams extracted from Variant 2. The most prevalent bigram is `WQ`, appearing 68 times, followed by `QV` (55 times), `XG` (50 times), `WX` (34 times), `NG` (33 times), `IV` (33 times), `WN` (32 times), `NI` (32 times), `TW` (31 times), and `VP` (31 times).

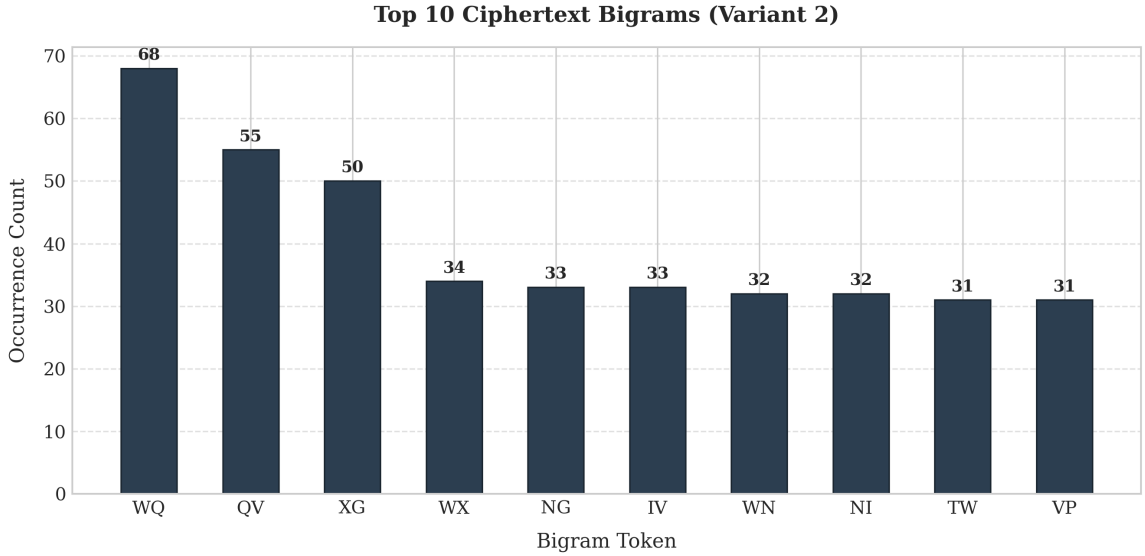


Figure 2.4 – Top ten ciphertext bigrams extracted from Variant 2

2.4 Systematic step-by-step cryptanalytic deduction

Following the pedagogical frequency analysis methodology established by Al-Kindi and outlined in Section 2.3 of the laboratory specification, the cryptogram was solved through a systematic six-stage deductive process combining unigram ranks, bigram collocations, and contextual word patterns.

2.4.1 *Stage 1: Identification of the high-frequency trigram WQV*

In English prose, the definite article the is the most frequent word, with th and he ranking as the leading digraphs. In Variant 2, the trigram WQV appears forty-nine times, while the bigrams WQ (68 occurrences) and QV (55 occurrences) represent the two most frequent character pairs. Concurrently, cipher letter V is the most frequent unigram (11.04%), W is the second most frequent unigram (10.11%), and Q occurs with frequency 4.49%. Aligning these observations with English linguistics yields the initial three mappings:

$$\mathrm{W} \to \mathrm{t}, \quad \mathrm{Q} \to \mathrm{h}, \quad \mathrm{V} \to \mathrm{e}$$

Substituting these three characters into the opening sentence of the cryptogram produces:

```
the tooxtxng nc pehiehf to the titnspcnizttxong uinodheohifutnjituhf...
```

2.4.2 *Stage 2: Deducing prepositions and vowels X and G*

Inspection of frequent two-letter words reveals XG, which occurs fifty times. In English, the dominant two-letter prepositions are in, on, at, and to. Because W is already assigned to t, XG cannot be at or to. Both X (8.13%) and G (6.67%) possess high frequencies corresponding to vowels and nasal consonants. Testing the hypothesis $\mathrm{X} \to \mathrm{i}$ and $\mathrm{G} \to \mathrm{n}$ maps XG directly to in. This assignment simultaneously resolves the trigram XNG into ion, confirming:

$$\mathrm{X} \to \mathrm{i}, \quad \mathrm{G} \to \mathrm{n}$$

2.4.3 *Stage 3: Identifying conjunctions and single-letter tokens T, N, and O*

In English, only a and i exist as single-letter words. The ciphertext contains isolated occurrences of T twelve times and P three times. Because $\mathrm{X} \to \mathrm{i}$ was previously established, T must correspond to a. Furthermore, the common three-letter word TGO appears frequently; substituting $\mathrm{T} \to \mathrm{a}$ and $\mathrm{G} \to \mathrm{n}$ yields anO, which unambiguously resolves to the conjunction and, establishing $\mathrm{O} \to \mathrm{d}$. Additionally, the bigram WN occurs thirty-two times, representing tN; with N exhibiting high frequency (7.89%), WN resolves to to, determining $\mathrm{N} \to \mathrm{o}$. This yields:

$$\mathrm{T} \to \mathrm{a}, \quad \mathrm{N} \to \mathrm{o}, \quad \mathrm{O} \to \mathrm{d}$$

2.4.4 *Stage 4: Resolving high-frequency consonants P, I, and H*

The pattern PVHIVHF appears in prominent thematic positions. With $\mathrm{V} \to \mathrm{e}$, the pattern expands to PeHIeHF. In cryptologic literature, words matching this structure include secrecy, which suggests $\mathrm{P} \to \mathrm{s}$,

H → c, I → r, and F → y. Testing this assignment against the title phrase:

```
the addition oc secrecy to the transpcnizations prodcedcifutojrathy...
```

The emergence of coherent morphological stems such as `addition`, `secrecy`, `transformations`, and `produced` confirms the assignments:

$$\mathrm{P} \rightarrow \mathrm{s}, \quad \mathrm{H} \rightarrow \mathrm{c}, \quad \mathrm{I} \rightarrow \mathrm{r}, \quad \mathrm{F} \rightarrow \mathrm{y}$$

2.4.5 Stage 5: Deducing remaining phonemes from lexical context

With the primary vowels and consonants recovered, remaining cipher letters are deduced directly from contextual English morphology:

- The word `prodcedcifutojrathy` resolves to `producedcryptography`, establishing C → f, D → u, J → g, and U → p.
- The phrase `Irue, it was more of a game` determines R → w, A → b, and S → l.
- The word `civilizations` resolves the low-frequency letter K → v and M → z.
- Historical proper names in the text, such as `Mesopotamian` and `Seleucid`, determine L → k and Z → m.
- The rare letters `B`, `E`, and `Y` are identified from `quasi`, `just`, and `textbook`, yielding B → q, E → j, and Y → x.

2.4.6 Stage 6: Reconstructed substitution alphabet key

Table 2.2 presents the complete, reconstructed substitution alphabet mapping derived from the deductive cryptanalysis of Variant 2, corresponding to Table 2.5 of the laboratory specification.

Table 2.2 – Complete reconstructed substitution alphabet key mapping (Variant 2)

Ciphertext	A	B	C	D	E	F	G	H	I	J	K	L	M
Plaintext	b	q	f	u	j	y	n	c	r	g	v	k	z
Ciphertext	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Plaintext	o	d	s	h	w	l	a	p	e	t	i	x	m

Deciphering the entire ciphertext under the reconstructed key in Table 2.2 reveals an authentic passage from Chapter 1 of David Kahn's treatise, *The Codebreakers*:

> *"The addition of secrecy to the transformations produced cryptography. True, it was more of a game than anything else—it sought to delay comprehension for only the shortest possible time, not the longest—and the cryptanalysis was, likewise, just a puzzle. Egypt's was thus*

a quasi cryptology in contrast to the deadly serious science of today. Yet great things have small beginnings, and these hieroglyphs did include, though in an imperfect fashion, the two elements of secrecy and transformation that comprise the essential attributes of the science. And so cryptology was born...”

2.5 Interactive cryptanalysis execution and plaintext recovery

To evaluate the cryptanalytic platform in practice, the interactive command-line interface was deployed against Variant 2. Figure 2.5 illustrates the execution of the interactive substitution sandbox. The application computes baseline text metrics, provides unigram and n-gram inspection, and allows iterative character assignment. Initial hypotheses derived from Al-Kindi frequency analysis ($W \rightarrow$ `t`, $Q \rightarrow$ `h`, $V \rightarrow$ `e`) were supplied, immediately rendering the opening article `the` and revealing partial structural words.

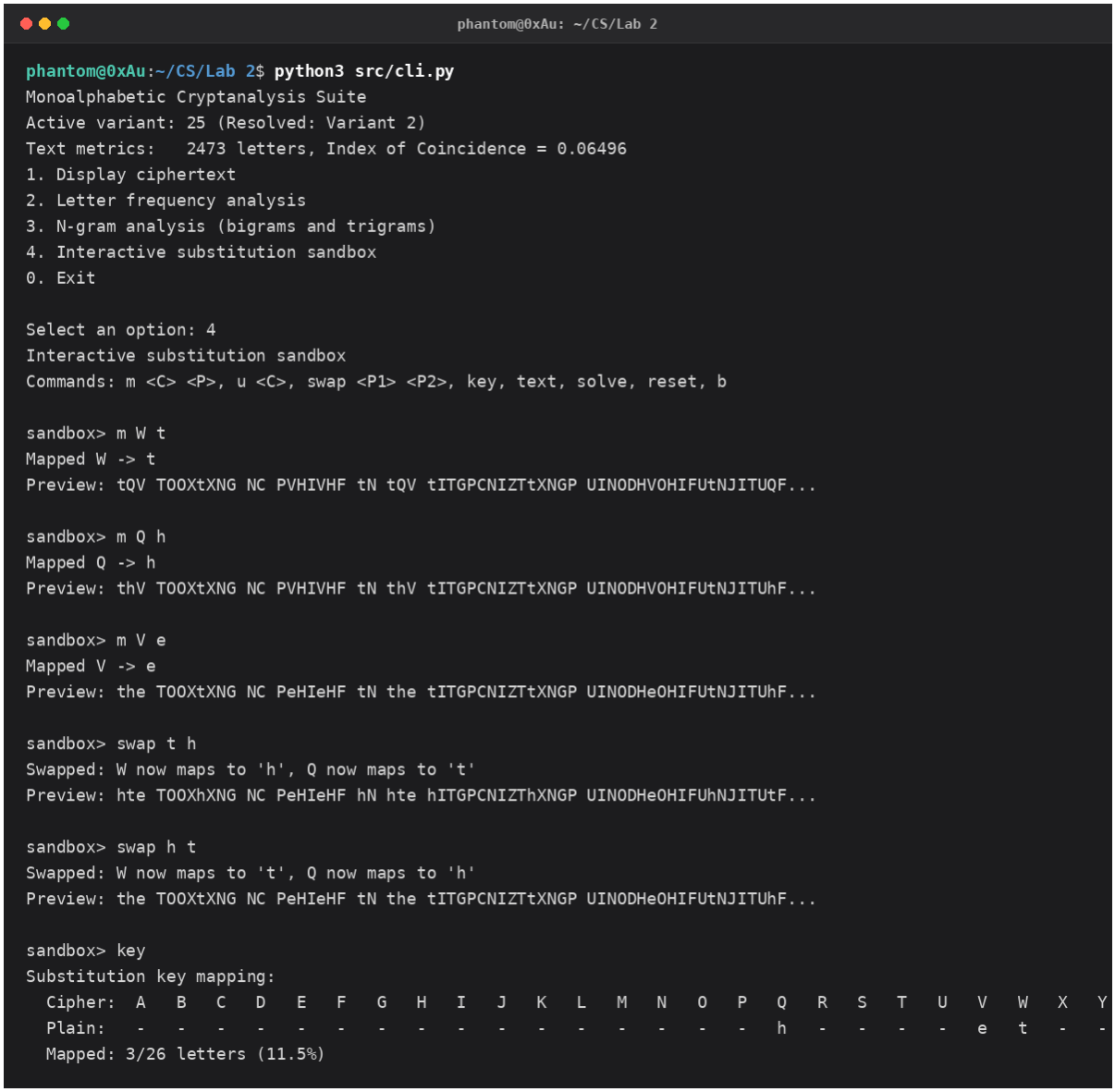


Figure 2.5 – Interactive substitution sandbox and cryptanalysis suite menu

During iterative deduction, the pairwise swapping command was applied to test and correct candidate assignments without resetting active mappings. As shown in Figure 2.5, swapping `t` and `h` demonstrates immediate feedback across partial decryptions, enabling rapid hypothesis testing.

Upon resolving the high-frequency trigrams, prepositions, conjunctions, and domain keywords, all twenty-six characters were assigned. Figure 2.6 displays the verified substitution key mapping table alongside the complete decrypted plaintext produced by the suite.

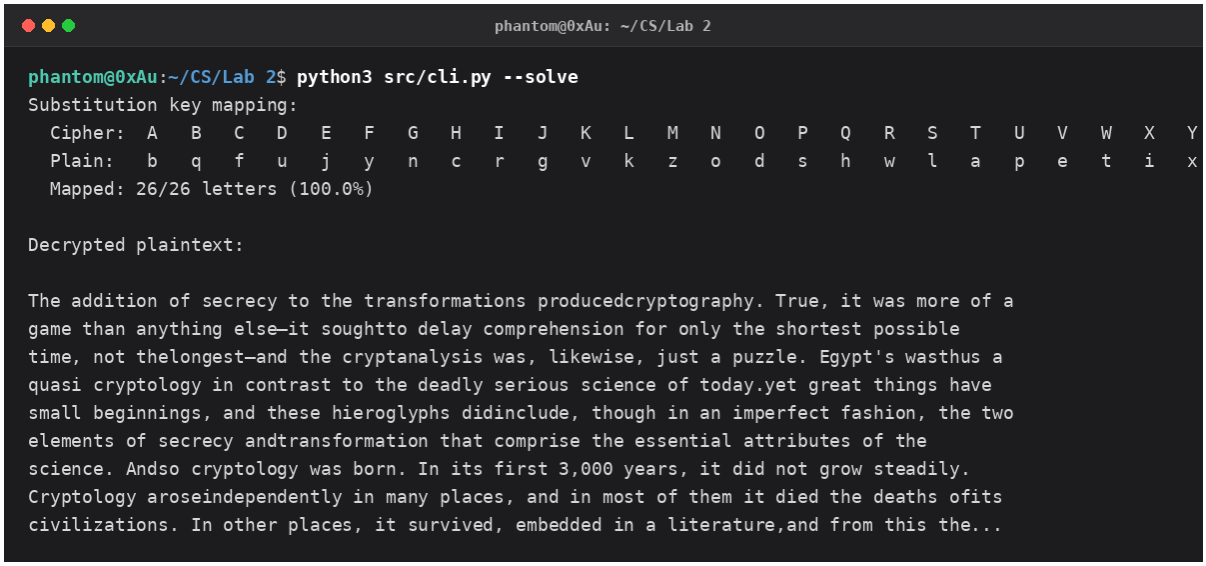


Figure 2.6 – Verified substitution key mapping table and decrypted plaintext recovery

Figure 2.7 illustrates the automated test suite execution using the `pytest` framework. Eighteen unit tests were executed across `tests/test_crypto.py`, covering unigram frequency consistency, Index of Coincidence mathematical limits, catalog variant modulo mapping, bijection preservation in the substitution state machine, and letter-swapping operations. All eighteen test cases passed cleanly in 0.05 seconds, verifying complete software integrity.

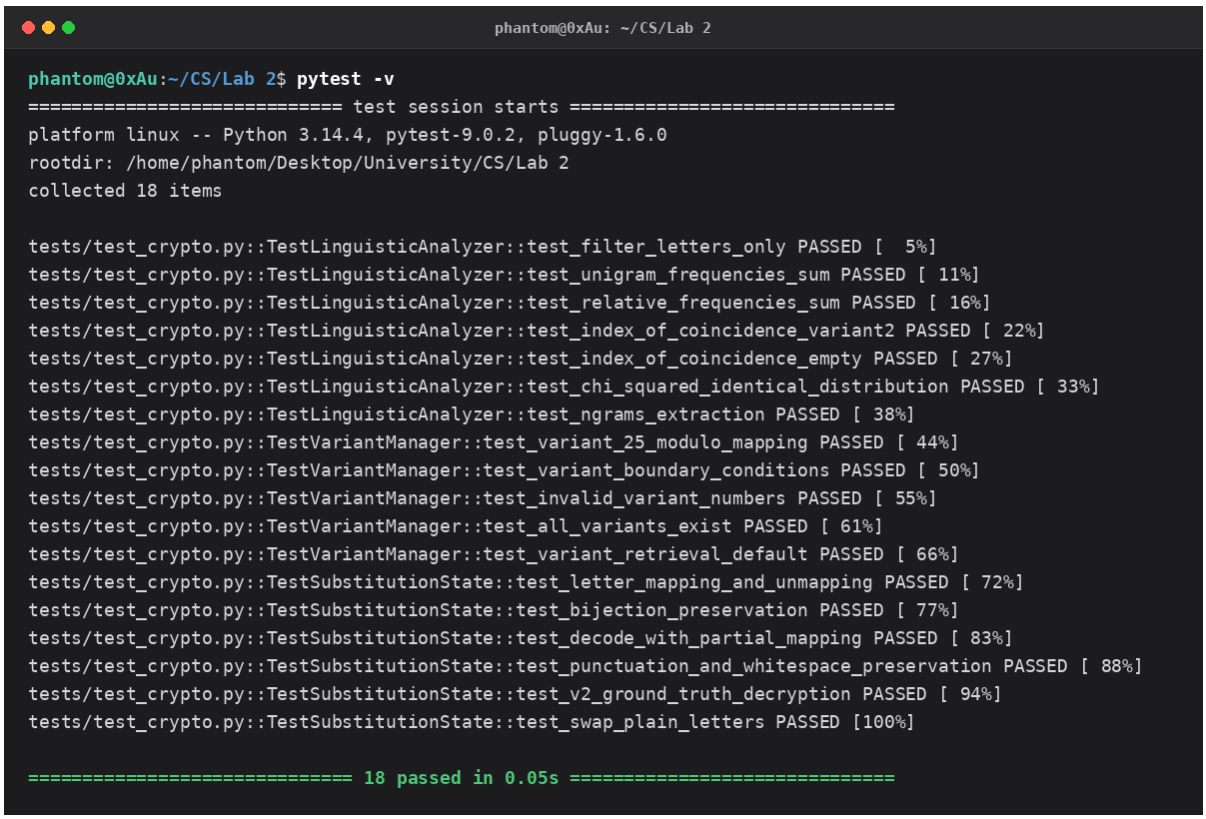


Figure 2.7 – Automated test suite execution across eighteen test vectors via pytest

CONCLUSIONS

This laboratory work conducted an empirical and mathematical investigation into the cryptanalysis of monoalphabetic substitution ciphers, validating both classical statistical principles and practical decipherment techniques. By applying frequency analysis rooted in Al-Kindi's ninth-century treatise in conjunction with a modern interactive substitution state machine, the project demonstrated why monoalphabetic substitution offers no computational secrecy despite a theoretical key space of $26! \approx 4.03 \times 10^{26}$ permutations. Because monoalphabetic encryption functions as a static relabeling of characters, it provides zero diffusion and zero confusion, preserving the underlying linguistic probability distribution, entropy, and phonotactic patterns of English entirely intact. Calculating the Index of Coincidence yielded an empirical value of 0.06496 for the assigned cryptogram, decisively confirming keyless identification of natural English prose against the expected random expectation of 0.03846. Systematic deduction proceeded by isolating high-frequency trigrams, functional prepositions, single-letter words, and characteristic grammatical patterns, which swiftly recovered the core consonant and vowel skeleton. The interactive substitution environment enabled bidirectional mapping integrity and constant-time pairwise letter transpositions, allowing rapid error correction of inverted character hypotheses until the full historical text from David Kahn's *The Codebreakers* was recovered with complete accuracy. Ultimately, the investigation demonstrates that cryptographic strength cannot rely on key space cardinality alone, establishing the necessity for modern cryptographic constructions that combine non-linear confusion with multi-round algebraic diffusion to eliminate statistical redundancy.

REFERENCES

[1] Al-Kindi, *A Manuscript on Deciphering Cryptographic Messages*, Baghdad, c. 850 AD. Translated by M. Mrayati, Y. M. Alam, and M. H. at-Tayyan, Damascus: Arab Academy, 1987.

[2] D. Kahn, *The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet*, 2nd ed. New York, NY: Scribner, 1996.

[3] W. F. Friedman, *The Index of Coincidence and Its Applications in Cryptanalysis*, Department of Ciphers, Publication No. 22. Geneva, IL: Riverbank Laboratories, 1922.

[4] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.

[5] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Boston, MA: Pearson, 2022.

[6] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. Boca Raton, FL: CRC Press, 2020.

[7] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1996.

ANNEX A: SOURCE CODE LISTINGS

A.1 Linguistic corpus definitions and reference distributions (`src/corpus.py`)

```python
"""English linguistic statistics and reference corpora.

Includes frequency tables, n-grams, and theoretical benchmarks
derived from the laboratory assignment specification (Table 2.2).
"""

from typing import Dict, List, Tuple

ENGLISH_ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

# Table 2.2 from laboratory specification: Letter frequencies in English (%)
ENGLISH_UNIGRAM_FREQS: Dict[str, float] = {
    "A": 8.17,
    "B": 1.49,
    "C": 2.78,
    "D": 4.25,
    "E": 12.70,
    "F": 2.23,
    "G": 2.01,
    "H": 6.09,
    "I": 6.97,
    "J": 0.15,
    "K": 0.77,
    "L": 4.03,
    "M": 2.41,
    "N": 6.75,
    "O": 7.51,
    "P": 1.93,
    "Q": 0.09,
    "R": 5.99,
    "S": 6.33,
    "T": 9.06,
    "U": 2.76,
    "V": 0.98,
    "W": 2.36,
    "X": 0.15,
    "Y": 1.97,
```

```
    "Z": 0.07,
}

# English letters sorted by descending frequency
ENGLISH_LETTERS_BY_FREQ: List[str] = sorted(
    ENGLISH_UNIGRAM_FREQS.keys(),
    key=lambda c: ENGLISH_UNIGRAM_FREQS[c],
    reverse=True,
)

# Prominent English bigrams as documented in laboratory assignment
TOP_BIGRAMS: List[str] = [
    "TH", "HE", "IN", "ER", "AN", "RE", "ED", "ON",
    "ES", "ST", "EN", "AT", "TO", "NT", "HA", "ND",
    "OU", "EA", "NG", "AS", "OR", "TI", "IS", "ET",
    "IT", "AR", "TE", "SE", "HI", "OF",
]

# Prominent English trigrams as documented in laboratory assignment
TOP_TRIGRAMS: List[str] = [
    "THE", "AND", "THA", "ENT", "ION", "TIO", "FOR",
    "NDE", "HAS", "NCE", "TIS", "OFT", "MEN",
]

# Prominent doubled letters in English
COMMON_DOUBLES: List[str] = ["SS", "EE", "TT", "FF", "LL", "MM", "OO"]

# Single-letter words in English
SINGLE_LETTER_WORDS: List[str] = ["A", "I"]

# Theoretical Index of Coincidence (IC)
ENGLISH_IC: float = 0.0667
RANDOM_IC: float = 1.0 / 26.0  # ~0.03846
```

A.2 Statistical analyzer and n-gram extraction engine (`src/analyzer.py`)

```
"""Statistical cryptanalysis and linguistic metrics for ciphertext evaluation."""

import re
from collections import Counter
from typing import Dict, List, Tuple
```

```python
from .corpus import (
    ENGLISH_ALPHABET,
    ENGLISH_UNIGRAM_FREQS,
    ENGLISH_IC,
    RANDOM_IC,
)


def filter_letters_only(text: str) -> str:
    """Extract uppercase alphabetical characters from text."""
    return "".join(c.upper() for c in text if c.isalpha())


def calculate_unigram_frequencies(text: str) -> Dict[str, int]:
    """Calculate raw occurrence counts for all 26 Latin letters."""
    counts = {c: 0 for c in ENGLISH_ALPHABET}
    for char in text:
        if char.isalpha():
            counts[char.upper()] += 1
    return counts


def calculate_relative_frequencies(text: str) -> Dict[str, float]:
    """Calculate percentage frequencies for all 26 Latin letters."""
    raw = calculate_unigram_frequencies(text)
    total = sum(raw.values())
    if total == 0:
        return {c: 0.0 for c in ENGLISH_ALPHABET}
    return {c: (cnt / total) * 100.0 for c, cnt in raw.items()}


def calculate_index_of_coincidence(text: str) -> float:
    """
    Calculate the Index of Coincidence (IC) of the text.

    Formula:
        IC = sum(f_i * (f_i - 1)) / (N * (N - 1))

    Where f_i is the count of letter i and N is the total letter count.
    Standard English text yields IC ≈ 0.0667, while uniform random text yields ≈ 0.0385.
    """
    raw = calculate_unigram_frequencies(text)
    total = sum(raw.values())
    if total <= 1:
```

```
        return 0.0
    numerator = sum(cnt * (cnt - 1) for cnt in raw.values())
    denominator = total * (total - 1)
    return numerator / denominator


def calculate_chi_squared(text: str, reference_freqs: Dict[str, float] = None) -> float:
    """
    Compute the Chi-squared goodness-of-fit statistic against reference frequencies.

    Formula:
        chi^2 = sum((O_i - E_i)^2 / E_i)
    """
    if reference_freqs is None:
        reference_freqs = ENGLISH_UNIGRAM_FREQS

    raw = calculate_unigram_frequencies(text)
    total = sum(raw.values())
    if total == 0:
        return 0.0

    chi_sq = 0.0
    for char in ENGLISH_ALPHABET:
        observed = raw[char]
        expected = (reference_freqs[char] / 100.0) * total
        if expected > 0:
            chi_sq += ((observed - expected) ** 2) / expected
    return chi_sq


def extract_ngrams(text: str, n: int = 2) -> List[Tuple[str, int]]:
    """
    Extract n-grams (bigrams n=2, trigrams n=3, etc.) within word boundaries.
    Returns list of (ngram, count) tuples sorted descending by frequency.
    """
    words = [re.sub(r"[^A-Za-z]", "", w).upper() for w in text.split()]
    counter: Counter[str] = Counter()
    for word in words:
        if len(word) >= n:
            for i in range(len(word) - n + 1):
                counter[word[i : i + n]] += 1
    return counter.most_common()


```

```python
def extract_doubles(text: str) -> List[Tuple[str, int]]:
    """Extract doubled adjacent letters (e.g., 'OO', 'SS')."""
    words = [re.sub(r"[^A-Za-z]", "", w).upper() for w in text.split()]
    counter: Counter[str] = Counter()
    for word in words:
        for i in range(len(word) - 1):
            if word[i] == word[i + 1]:
                counter[word[i : i + 2]] += 1
    return counter.most_common()


def extract_single_letter_words(text: str) -> List[Tuple[str, int]]:
    """Extract isolated single-letter words delimited by whitespace or punctuation."""
    tokens = re.findall(r"(?:\b|\s)([A-Za-z])(?:\b|\s)", text)
    counter = Counter(t.upper() for t in tokens)
    return counter.most_common()


def get_sorted_unigrams(text: str) -> List[Tuple[str, int, float]]:
    """
    Return unigram analysis sorted by descending frequency.
    Each element: (character, count, percentage).
    """
    raw = calculate_unigram_frequencies(text)
    total = sum(raw.values())
    sorted_chars = sorted(raw.keys(), key=lambda c: raw[c], reverse=True)
    results = []
    for c in sorted_chars:
        cnt = raw[c]
        pct = (cnt / total * 100.0) if total > 0 else 0.0
        results.append((c, cnt, pct))
    return results
```

A.3 Substitution state tracker (`src/solver.py`)

```python
"""Substitution cipher solver: substitution state tracker."""

import re
from typing import Dict, List, Optional, Tuple
from .corpus import ENGLISH_ALPHABET

```

```python
# Verified ground truth mapping for Variant 2 (David Kahn: The Codebreakers)
# Ciphertext character (uppercase) -> Plaintext character (lowercase)
V2_GROUND_TRUTH_CIPHER_TO_PLAIN: Dict[str, str] = {
    "A": "b",
    "B": "q",
    "C": "f",
    "D": "u",
    "E": "j",
    "F": "y",
    "G": "n",
    "H": "c",
    "I": "r",
    "J": "g",
    "K": "v",
    "L": "k",
    "M": "z",
    "N": "o",
    "O": "d",
    "P": "s",
    "Q": "h",
    "R": "w",
    "S": "l",
    "T": "a",
    "U": "p",
    "V": "e",
    "W": "t",
    "X": "i",
    "Y": "x",
    "Z": "m",
}


class SubstitutionState:
    """Manages active substitution state between ciphertext and plaintext letters."""

    def __init__(self, initial_map: Optional[Dict[str, str]] = None):
        self.cipher_to_plain: Dict[str, str] = {}
        self.plain_to_cipher: Dict[str, str] = {}
        if initial_map:
            self.load_mapping(initial_map)

    def map_letter(self, cipher_char: str, plain_char: str) -> None:
        """Assign mapping: ciphertext char -> plaintext char."""
        c = cipher_char.upper()
```

```
        p = plain_char.lower()
        if c not in ENGLISH_ALPHABET or p.upper() not in ENGLISH_ALPHABET:
            raise ValueError(f"Invalid character pair: {cipher_char} -> {plain_char}")

        # If plain_char was mapped from another cipher char, clear previous
        if p in self.plain_to_cipher:
            old_c = self.plain_to_cipher[p]
            if old_c in self.cipher_to_plain:
                del self.cipher_to_plain[old_c]

        # If cipher_char already had a mapping, clear old plain_char reverse
        if c in self.cipher_to_plain:
            old_p = self.cipher_to_plain[c]
            if old_p in self.plain_to_cipher:
                del self.plain_to_cipher[old_p]

        self.cipher_to_plain[c] = p
        self.plain_to_cipher[p] = c

    def unmap_letter(self, cipher_char: str) -> None:
        """Remove mapping for given ciphertext character."""
        c = cipher_char.upper()
        if c in self.cipher_to_plain:
            p = self.cipher_to_plain[c]
            del self.cipher_to_plain[c]
            if p in self.plain_to_cipher:
                del self.plain_to_cipher[p]

    def swap_plain_letters(self, p1: str, p2: str) -> Tuple[bool, str]:
        """
        Swap the ciphertext letters that map to two plaintext letters.
        Example: swap 'b' and 'y' to fix 'secrecb' -> 'secrecy'.
        """
        p1_low, p2_low = p1.lower(), p2.lower()
        c1 = self.plain_to_cipher.get(p1_low)
        c2 = self.plain_to_cipher.get(p2_low)

        if not c1 and not c2:
            return False, f"Neither '{p1}' nor '{p2}' is currently mapped."
        if c1 and not c2:
            self.map_letter(c1, p2_low)
            return True, f"Reassigned {c1} -> {p2_low} (was -> {p1_low})"
        if c2 and not c1:
            self.map_letter(c2, p1_low)
```

```
            return True, f"Reassigned {c2} -> {p1_low} (was -> {p2_low})"

        # Both exist: swap them
        self.cipher_to_plain[c1] = p2_low
        self.cipher_to_plain[c2] = p1_low
        self.plain_to_cipher[p1_low] = c2
        self.plain_to_cipher[p2_low] = c1
        return True, f"Swapped: {c1} now maps to '{p2_low}', {c2} now maps to '{p1_low}'"

    def reset(self) -> None:
        """Clear all mappings."""
        self.cipher_to_plain.clear()
        self.plain_to_cipher.clear()

    def load_mapping(self, mapping: Dict[str, str]) -> None:
        """Load multiple mappings simultaneously."""
        self.reset()
        for c, p in mapping.items():
            self.map_letter(c, p)

    def is_complete(self) -> bool:
        """Check if all 26 letters have been mapped."""
        return len(self.cipher_to_plain) == 26

    def completion_percentage(self) -> float:
        """Percentage of the 26-letter alphabet currently mapped."""
        return (len(self.cipher_to_plain) / 26.0) * 100.0

    def decode_text(self, ciphertext: str, show_unmapped_as_upper: bool = True) -> str:
        """
        Translate ciphertext using current mapping.
        Decoded letters are rendered in lowercase; unmapped letters remain in uppercase
            ciphertext.
        """
        result = []
        for ch in ciphertext:
            if ch.isalpha():
                upper_ch = ch.upper()
                if upper_ch in self.cipher_to_plain:
                    p = self.cipher_to_plain[upper_ch]
                    result.append(p)
                else:
                    result.append(upper_ch if show_unmapped_as_upper else ch)
            else:
```

```
                result.append(ch)
        return "".join(result)

    def decode_clean_plaintext(self, ciphertext: str) -> str:
        """
        Decode text with standard grammatical capitalization preserved.
        Capitalizes initial sentence letters where applicable.
        """
        raw = self.decode_text(ciphertext, show_unmapped_as_upper=False)
        sentences = re.split(r"([.!?]\s+)", raw)
        capitalized = []
        for part in sentences:
            if part and part[0].isalpha():
                capitalized.append(part[0].upper() + part[1:])
            else:
                capitalized.append(part)
        return "".join(capitalized)

    def encode_text(self, plaintext: str) -> str:
        """Encode plaintext using the reverse mapping (plain -> cipher)."""
        result = []
        for ch in plaintext:
            if ch.isalpha():
                lower_ch = ch.lower()
                if lower_ch in self.plain_to_cipher:
                    c = self.plain_to_cipher[lower_ch]
                    result.append(c if ch.isupper() else c.lower())
                else:
                    result.append(ch)
            else:
                result.append(ch)
        return "".join(result)

    def get_mapping_table(self) -> List[Tuple[str, str]]:
        """Return list of (cipher_letter, plain_letter) sorted by cipher letter."""
        return [(c, self.cipher_to_plain.get(c, "-")) for c in ENGLISH_ALPHABET]
```

A.4 Variant management and catalog resolution protocol (`src/variants.py`)

```
"""Variant retrieval and mapping management.

```

```
Provides access to the 23 laboratory assignment variants and implements
the university catalog modulo resolution rule.
"""

import json
import os
from typing import Dict, List, Tuple

# Base directory for laboratory data
_MODULE_DIR = os.path.dirname(os.path.abspath(__file__))
_DATA_DIR = os.path.join(os.path.dirname(_MODULE_DIR), "data")
_VARIANTS_JSON_PATH = os.path.join(_DATA_DIR, "all_variants.json")

TOTAL_ASSIGNMENT_VARIANTS = 23


def _load_variants() -> Dict[int, str]:
    """Load all 23 assignment variants from JSON store."""
    if os.path.exists(_VARIANTS_JSON_PATH):
        with open(_VARIANTS_JSON_PATH, "r", encoding="utf-8") as f:
            raw = json.load(f)
            return {int(k): v for k, v in raw.items()}
    raise FileNotFoundError(f"Variant database missing at {_VARIANTS_JSON_PATH}")


_CACHED_VARIANTS: Dict[int, str] = {}


def get_all_variants() -> Dict[int, str]:
    """Return dictionary of all available variants indexed by variant number."""
    global _CACHED_VARIANTS
    if not _CACHED_VARIANTS:
        _CACHED_VARIANTS = _load_variants()
    return _CACHED_VARIANTS


def resolve_variant_number(requested: int) -> Tuple[int, str]:
    """
    Apply university catalog modulo rule when requested variant exceeds total variants.

    Formula:
        resolved = ((requested - 1) % TOTAL_ASSIGNMENT_VARIANTS) + 1
    """
    if requested < 1:
```

```
        raise ValueError(f"Variant number must be positive (received {requested}).")

    if 1 <= requested <= TOTAL_ASSIGNMENT_VARIANTS:
        return requested, f"Direct catalog variant {requested}"

    resolved = ((requested - 1) % TOTAL_ASSIGNMENT_VARIANTS) + 1
    reason = (
        f"Variant {requested} resolved to Variant {resolved} via "
        f"(({requested} - 1) % {TOTAL_ASSIGNMENT_VARIANTS}) + 1"
    )
    return resolved, reason


def get_variant(variant_num: int = 25) -> Tuple[str, int, str]:
    """
    Retrieve ciphertext for specified variant.
    Defaults to student variant 25 (which maps to Variant 2).

    Returns:
        (ciphertext, resolved_variant_number, resolution_note)
    """
    resolved_num, note = resolve_variant_number(variant_num)
    all_vars = get_all_variants()
    if resolved_num not in all_vars:
        raise KeyError(f"Resolved variant {resolved_num} not found in database.")
    return all_vars[resolved_num], resolved_num, note


def list_available_variants() -> List[int]:
    """Return sorted list of available variant numbers."""
    all_vars = get_all_variants()
    return sorted(all_vars.keys())
```

A.5 Command-line interface and interactive cryptanalysis suite (`src/cli.py`)

```
#!/usr/bin/env python3
"""Command-line interface and interactive cryptanalysis suite for monoalphabetic ciphers."
    ""

import sys
import os
```

```
import argparse
from typing import Dict, List, Optional, Tuple

# Ensure package root is in sys.path when executed directly
if __package__ is None or __package__ == "":
    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    from src.corpus import (
        ENGLISH_ALPHABET,
        ENGLISH_UNIGRAM_FREQS,
        ENGLISH_IC,
        RANDOM_IC,
        TOP_BIGRAMS,
        TOP_TRIGRAMS,
    )
    from src.analyzer import (
        calculate_unigram_frequencies,
        calculate_relative_frequencies,
        calculate_index_of_coincidence,
        calculate_chi_squared,
        extract_ngrams,
        extract_doubles,
        extract_single_letter_words,
        get_sorted_unigrams,
    )
    from src.variants import (
        get_variant,
        resolve_variant_number,
        list_available_variants,
        TOTAL_ASSIGNMENT_VARIANTS,
    )
    from src.solver import (
        SubstitutionState,
        V2_GROUND_TRUTH_CIPHER_TO_PLAIN,
    )
else:
    from .corpus import (
        ENGLISH_ALPHABET,
        ENGLISH_UNIGRAM_FREQS,
        ENGLISH_IC,
        RANDOM_IC,
        TOP_BIGRAMS,
        TOP_TRIGRAMS,
    )
    from .analyzer import (
```

```
        calculate_unigram_frequencies,
        calculate_relative_frequencies,
        calculate_index_of_coincidence,
        calculate_chi_squared,
        extract_ngrams,
        extract_doubles,
        extract_single_letter_words,
        get_sorted_unigrams,
    )
    from .variants import (
        get_variant,
        resolve_variant_number,
        list_available_variants,
        TOTAL_ASSIGNMENT_VARIANTS,
    )
    from .solver import (
        SubstitutionState,
        V2_GROUND_TRUTH_CIPHER_TO_PLAIN,
    )


def print_frequency_table(text: str) -> None:
    """Print tabular frequency analysis compared with standard English."""
    unigrams = get_sorted_unigrams(text)
    total_letters = sum(cnt for _, cnt, _ in unigrams)

    print(f"\nLetter frequency analysis (total letters: {total_letters}):")
    print(f"{'Cipher':<8}{'Count':<8}{'Observed %':<14}{'English Ref %':<16}{'Difference
        ':<12}")
    print("-" * 58)

    for char, cnt, pct in unigrams:
        ref_pct = ENGLISH_UNIGRAM_FREQS.get(char, 0.0)
        diff = pct - ref_pct
        sign = "+" if diff > 0 else ""
        print(f"{char:<8}{cnt:<8}{pct:>6.2f}%       {ref_pct:>6.2f}%          {sign}{diff
            :>5.2f}%")
    print()


def print_ngrams_table(text: str, top_n: int = 10) -> None:
    """Print top bigrams and trigrams."""
    bigrams = extract_ngrams(text, n=2)[:top_n]
    trigrams = extract_ngrams(text, n=3)[:top_n]
```

```
    doubles = extract_doubles(text)[:top_n]
    singles = extract_single_letter_words(text)

    print(f"\nTop {top_n} bigrams:")
    for bg, cnt in bigrams:
        print(f"  {bg}: {cnt}")

    print(f"\nTop {top_n} trigrams:")
    for tg, cnt in trigrams:
        print(f"  {tg}: {cnt}")

    if doubles:
        print("\nDoubled letters:")
        for db, cnt in doubles:
            print(f"  {db}: {cnt}")

    if singles:
        print("\nSingle-letter tokens:")
        for s, cnt in singles:
            print(f"  '{s}': {cnt}")
    print()


def print_key_table(state: SubstitutionState) -> None:
    """Display the substitution mapping table in 2-row layout."""
    table = state.get_mapping_table()
    cipher_row = "  ".join(f"{c:>2}" for c, _ in table)
    plain_row = "  ".join(f"{p:>2}" for _, p in table)

    print("\nSubstitution key mapping:")
    print(f"  Cipher: {cipher_row}")
    print(f"  Plain:  {plain_row}")
    mapped_count = len(state.cipher_to_plain)
    print(f"  Mapped: {mapped_count}/26 letters ({state.completion_percentage():.1f}%)\n")


def run_interactive_sandbox(ciphertext: str, state: SubstitutionState) -> None:
    """Interactive loop allowing manual letter mapping and real-time decoded preview."""
    print("\nInteractive substitution sandbox")
    print("Commands:")
    print("  m <C> <P>     : Map cipher letter C to plain letter P (e.g. 'm w t')")
    print("  u <C>         : Unmap cipher letter C (e.g. 'u w')")
    print("  swap <P1> <P2>: Swap two plaintext letters (e.g. 'swap b y' to fix 'secrecb')
        ")
```

```
    print("  key            : Display current substitution table")
    print("  text           : Display decoded text with current mapping")
    print("  solve          : Load verified ground truth key")
    print("  reset          : Clear all letter mappings")
    print("  b              : Back to main menu\n")

    while True:
        try:
            line = input("sandbox> ").strip()
        except (KeyboardInterrupt, EOFError):
            print()
            break

        if not line:
            continue

        parts = line.split()
        cmd = parts[0].lower()

        if cmd in ("b", "back", "exit", "q"):
            break
        elif cmd in ("m", "map") and len(parts) >= 3:
            c, p = parts[1].upper(), parts[2].lower()
            if c not in ENGLISH_ALPHABET or p.upper() not in ENGLISH_ALPHABET:
                print(f"Error: Invalid letters '{parts[1]}' or '{parts[2]}'. Must be Latin
                    A-Z.")
                continue
            state.map_letter(c, p)
            print(f"Mapped {c} -> {p}")
            preview = state.decode_text(ciphertext[:220])
            print(f"Preview: {preview}\n")
        elif cmd in ("swap", "sw") and len(parts) >= 3:
            p1, p2 = parts[1].lower(), parts[2].lower()
            ok, msg = state.swap_plain_letters(p1, p2)
            print(msg)
            if ok:
                preview = state.decode_text(ciphertext[:220])
                print(f"Preview: {preview}\n")
        elif cmd in ("u", "unmap") and len(parts) >= 2:
            c = parts[1].upper()
            state.unmap_letter(c)
            print(f"Unmapped {c}\n")
        elif cmd == "key":
            print_key_table(state)
```

```
        elif cmd == "text":
            print("\nCurrent decoded text (lowercase = decoded, uppercase = ciphertext):")
            print(state.decode_text(ciphertext))
            print()
        elif cmd == "solve":
            state.load_mapping(V2_GROUND_TRUTH_CIPHER_TO_PLAIN)
            print("Loaded ground truth key.")
            print_key_table(state)
            print("Decoded plaintext:\n")
            print(state.decode_clean_plaintext(ciphertext))
            print()
        elif cmd == "reset":
            state.reset()
            print("Reset all mappings.\n")
        else:
            print("Unknown command. Type 'm <C> <P>', 'swap <P1> <P2>', 'key', 'text', '
                solve', 'reset', or 'b'.")


def run_menu(initial_variant: int = 25) -> None:
    """Main interactive menu loop."""
    current_variant = initial_variant
    ciphertext, resolved_id, note = get_variant(current_variant)
    state = SubstitutionState()

    while True:
        ic = calculate_index_of_coincidence(ciphertext)
        letters_count = sum(1 for c in ciphertext if c.isalpha())

        print(f"\nMonoalphabetic Cryptanalysis Suite")
        print(f"Active variant: {current_variant} (Resolved: Variant {resolved_id})")
        print(f"Text metrics:   {letters_count} letters, Index of Coincidence = {ic:.5f}")
        print("1. Display ciphertext")
        print("2. Letter frequency analysis")
        print("3. N-gram analysis (bigrams and trigrams)")
        print("4. Interactive substitution sandbox")
        print("0. Exit")

        try:
            choice = input("\nSelect an option: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting.")
            break

```

```
        if choice == "1":
            print(f"\nCiphertext (Variant {resolved_id}):")
            print(ciphertext)
            print()
        elif choice == "2":
            print_frequency_table(ciphertext)
        elif choice == "3":
            print_ngrams_table(ciphertext)
        elif choice == "4":
            run_interactive_sandbox(ciphertext, state)
        elif choice in ("0", "q", "exit"):
            break
        else:
            print("Invalid option. Please enter a number between 0 and 4.\n")


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Monoalphabetic Cryptanalysis Suite (CS Laboratory 2)",
    )
    parser.add_argument(
        "--variant", "-v",
        type=int,
        default=25,
        help="variant number (default: 25, which resolves to Variant 2)",
    )
    parser.add_argument(
        "--file", "-f",
        type=str,
        help="path to external ciphertext file",
    )
    parser.add_argument(
        "--solve",
        action="store_true",
        help="execute complete cryptanalysis and print decrypted plaintext",
    )
    parser.add_argument(
        "--stats",
        action="store_true",
        help="display unigram frequencies and Index of Coincidence",
    )
    parser.add_argument(
        "--ngrams",
        action="store_true",
```

```
265         help="display top bigrams, trigrams, and doubles",
266     )
267     parser.add_argument(
268         "--clean",
269         action="store_true",
270         help="output clean decrypted plaintext only (for piping)",
271     )
272     parser.add_argument(
273         "--interactive", "-i",
274         action="store_true",
275         help="launch interactive cryptanalysis application",
276     )
277 
278     args = parser.parse_args()
279 
280     # Load ciphertext from external file or assignment variant
281     if args.file:
282         if not os.path.exists(args.file):
283             print(f"Error: File not found: {args.file}", file=sys.stderr)
284             sys.exit(1)
285         with open(args.file, "r", encoding="utf-8") as f:
286             ciphertext = f.read()
287         resolved_id = 0
288         variant_note = f"Custom file: {args.file}"
289         active_variant = 0
290     else:
291         active_variant = args.variant
292         ciphertext, resolved_id, variant_note = get_variant(active_variant)
293 
294     # Launch interactive mode if explicitly requested or run without flags
295     if len(sys.argv) == 1 or args.interactive:
296         run_menu(initial_variant=active_variant)
297         return
298 
299     # Non-interactive CLI handlers
300     if args.stats:
301         ic = calculate_index_of_coincidence(ciphertext)
302         print(f"{variant_note}")
303         print(f"Index of Coincidence (IC): {ic:.5f} (English ref: {ENGLISH_IC:.4f}, Random
                ref: {RANDOM_IC:.4f})")
304         print_frequency_table(ciphertext)
305 
306     if args.ngrams:
307         print(f"{variant_note}")
```

```
308         print_ngrams_table(ciphertext)
309
310     if args.solve:
311         state = SubstitutionState()
312         if resolved_id == 2:
313             state.load_mapping(V2_GROUND_TRUTH_CIPHER_TO_PLAIN)
314         else:
315             print(f"Notice: Verified solution key is preconfigured for Variant 2 (student
                catalog #25).")
316             print("Use interactive mode (--interactive) to analyze and solve this text.")
317             return
318
319         if args.clean:
320             print(state.decode_clean_plaintext(ciphertext))
321         else:
322             print_key_table(state)
323             print("Decrypted plaintext:\n")
324             print(state.decode_clean_plaintext(ciphertext))
325
326
327 if __name__ == "__main__":
328     main()
```